1 2 9 0 FACULDADE DE CIÊNCIAS E TECNOLOGIA UNIVERSIDADE D COIMBRA

Chasing Phishing URLs: a Machine Learning approach to classify web addresses

Project Report

Guilherme Soares das Neves Cardoso
2021226995

Guilherme de Moniz e Henriques
2020237352

Machine Learning In Biology

2024/25

Index

Introduction

In this day and age, there have been major advancements in all types of online services, which also led to a drastic increase in the user's exposure to cyber threats/attacks, and out of all of the existing types, phishing still remains one of the most prevalent ones. Phishing attacks typically consist in sending deceptive messages to users, often in the form of personal messages (SMS) or emails, that tempt them to click malicious URLs designed to steal and get access to your personal, financial, or authentication information. Despite the growing awareness, these attacks continue to affect a lot of people everyday, exploiting human vulnerability and technological loopholes. Due to these issues and for these reasons, cybersecurity research has become essential to develop automated tools to detect phishing URLs.

The objective of this study is, the development and evaluation of machine learning (ML) classifiers that can automatically identify phishing URLs, using the PhiUSIL Phishing URL Dataset, compiling URLs, with a mix of both legitimate and phishing examples. Having more than 50 features, derived from the raw URL strings, as well as from the HTML source code. Additionally, including hyperparameters such as: URLCharProb and TLDLegitimateProb, which gather statistical patterns of higher order. These types of attributes provide an essential base for training efficient and effective classifiers, while also introducing challenges when it comes to feature redundancy, requiring more advanced feature selection and dimensionality reduction methods.

Throughout our work we took a systematic approach to both preprocessing and feature selection/engineering. Feature scaling was applied to ensure numerical stability followed by a dimensionality reduction using Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA). and feature selection based on a correlation and importance ranking. Various types of classification models were then implemented and used to compare results, including a Minimum Distance classifier with Fisher's LDA, Random Forest, Support Vector Machines (SVM), and Multi Layer Perceptrons (MLP). To address more complex things within the dataset, performance metrics sensitive to class imbalances were used, such as F1-score, sensitivity, specificity and Area Under the Curve (AUC) ROC curve.

This project's purpose is not only to achieve a high classification performance, but also to analyse the different model's behaviour and understand the features and patterns that

contribute the most to detecting phishing URLs. Through statistical techniques and machine learning tools, this work contributes to the broader field of intelligent cyber security systems and pattern recognition.

Methodology

Programming language and Libraries

This project was developed using Python 3 and the following libraries were used: pandas, numpy, random, seaborn, matplotlib, sklearn, scipy, collections and imblearn. We added a random seed of 40 and random state of 40 (when applicable) to ensure reproducibility of the results.

Dataset

The dataset used was the PhiUSIL Phishing URL Dataset, which comprises 235.795 labeled URLs, of which, 134.850 are legitimate and 100.945 phishing. It also provides 55 features extracted from the URLs and the HTML content of the desired target pages, including both base and hyper parameters. This dataset exhibits a lot of diversity, a large sample size, and rich feature representation, making it suitable for both training and evaluating different classifiers for phishing detection.

Data Preprocessing

Data Cleaning

This initial and essential process consists in the verification and treatment of missing values, duplicated lines, and categorical features. As the dataset did not showcase significant null values nor invalid entries, it was possible to execute a minimal cleaning. All the duplicated lines were removed and the categorical features were discarded, focusing purely on the numerical variables. After this process was done, the final dataset kept only approximately 235.000 lines of the original ones.

Feature Distribution

Understanding the distribution of features is also a critical step in order to appropriately select the necessary preprocessing techniques. Histograms and box plots were used to evaluate the spread and skewness of numeric features. Many of the features weren’t

distributed normally, which contributed later on to our choice of non parametric tests for feature relevance analysis. Due to their largest size, the feature distribution histograms and the non-binary feature distribution by target class boxplots of the non binary features are attached in APPENDIX (end of the document).

Outlier Handling

The detection of outliers was done in an exploratory format, through boxplots and distribution graphs, focusing only on the non-binary features (meaning, with more than two distinct values). The identified outliers were replaced by the average of the non outlier values of the respective feature. This method allowed us to mitigate the impact of extreme values, without losing relevant informative structure.

Feature Correlation Analysis

The correlation analysis was done through a Pearson correlation matrix, with the objective of identifying and removing features that are highly correlated with each other (> 0.9), since these can introduce redundancy, and negatively affect model performance. Features with zero variance were also removed at this stage, since they do not provide any discrimination between classes. After this analysis, the number of features was reduced to 39.

Data Scaling

To ensure that all features contribute equally to all the models, especially the ones that are more sensitive to feature magnitude like KNN and SVM. A Min-Max scaling was applied, normalizing the values in the range [0, 1]. This technique preserves the original distribution of the data, which is beneficial since many features do not follow a normal distribution.

Class Balancing

Due to the imbalanced nature of the dataset (with more legitimate examples than phishing examples), oversampling was applied to the minority class (phishing) and undersampling to the majority class (legitimate), creating balanced substes for training the models. This balance allowed us to improve metrics sensitive to imbalance, such as F1-score and sensitivity without compromising the representativeness of the data.

Feature Selection & Reduction

PCA (Principal Component Analysis)

PCA is an unsupervised linear technique that projects data into orthogonal axes of maximum variance. It was applied in our work to reduce dimensionality while also preserving the variance structure. Although PCA doesn't consider class labels, it helps uncover latent structures and reduce noise, particularly beneficial for algorithms like SVM. Based on other works and following an empirical approach, we chose the number of components that explain 95% of the variance.

LDA (Linear Discriminant Analysis)

LDA is a supervised method that maximizes class separability by projecting data onto a "subspace" that maximizes the ratio of between class to within class variance. Unlike PCA, this one uses label information, making it particularly effective for classification tasks. Fisher's LDA was also used as a classifier, taking advantage of its linear decision boundary for two class problems.

Kruskal-Wallis Test

The Krustal Wallis test is a non parametric test used to identify features that significantly differ between the phishing and legitimate classes. This test is strong on non normal data and was chosen because many feature distributions deviated based on assumptions made from Gaussian. The null hypothesis (H0) assumes that the distributions of a certain feature are the same across both classes, while the alternative hypothesis assumes that at least one distribution is different. Features with p-values below the significance level of 0.05 were considered statistically significant and, therefore, included in the final feature set.

Classifiers

MDC (Minimum Distance Classifier)

This classifier assigns labels based on the smallest distance to a mean vector per class, in the feature space. Two distance metrics were tested: Euclidean distance and Mahalanobis distance, the latter being more robust in cases of heterogeneous variance between features. MDC was also applied to the data transformed by LDA, exploring the linear separability of the classes in this projected space.

Fisher-LDA

Beyond being a tool for dimensionality reduction, Fisher's LDA can also serve as a linear classifier. It makes a decision boundary based on maximising class discrimination in a reduced space. Its interpretability and simplicity makes it a valuable model for binary classification.

KNN (K-Nearest Neighbours)

KNN is a non parametric, instance based learner that classifies points based on the majority “vote” of their k closest neighbors. This method is particularly useful for capturing complex decision boundaries. However, it is sensitive to feature scaling, which justifies the need for PCA and standardization beforehand.

GNB (Gaussian Naive Bayes)

The GNB assumes feature independence and models each feature distribution as Gaussian. Despite its simplicity and naive assumptions, GNB performs well on high dimensional data, which is our case, and is also efficient. It serves as a contrast to more complex classifiers and provides useful insights into the value of probabilistic modeling for this task.

SVM (Support Vector Machine)

SVMs are powerful margin based classifiers that find the more optimal hyperplane separating classes in a high dimensional space. Using a radial basis function (RBF) kernel, SVMs can model non linear boundaries. They were chosen for their strong generalization capabilities, especially in more unbalanced datasets when paired with class weighting strategies.

Evaluation Metrics & Cross-Validation

To evaluate the performance of the models, we used Stratified K-Fold cross-validation, a technique that splits the data into K subsets (folds). The process is repeated K times so that each fold is used once as a validation set while the remaining folds are used for training the classifier. This approach is better than a simple train/test split, as it reduces the impact of data variability and provides a more accurate estimate of the model’s ability to generalize to unseen data.

We set $K = 5$, meaning that the performance was evaluated across 5 folds, with shuffling enabled. Shuffling is important because it ensures that the data is randomly distributed before splitting, helping prevent any pattern in the dataset from introducing any bias into the folds.

For evaluating the classifier's performance, we measured the following metrics: accuracy, F1-score, sensitivity, and specificity. We also stored all true and predicted labels across folds in order to compute the confusion matrix of each classifier at the end of the cross-validation. This allows a better understanding of the model's behavior in terms of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN), offering a more detailed evaluation for each classifier.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$F1\ Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

$$Sensitivity = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{TN + FP}$$

Each classifier was evaluated both on the original dataset (i.e., with all features included) and on the dataset after applying feature selection and/or dimensionality reduction methods. This allowed us to compare the performance of the models on the full feature set against their performance after removing or reducing the features, helping us to verify if these techniques contributed to improving the model's performance.

Hyperparameter Tuning

We also performed hyperparameter tuning of KNN and SVM. This was applied using GridSearchCV combined with Stratified 5-fold cross-validation. This process allowed us to find the best combination of hyperparameters based on F1-score. The F1 score was chosen as the main scoring metric because it provides a balanced measure between precision and recall, which is especially important when dealing with false positives and false negatives. Accuracy alone can be misleading in such cases, so F1 is a better way to select the best model.

For KNN, we initially used *n_neighbors = 3*. During optimization, we tested a range of odd values from 3 to 19 for the number of neighbors. The goal was to identify the optimal

number of neighbors that give the best F1-score, after the classifier was evaluated across all five folds.

For SVM, the default model used the rbf kernel with default regularization. In the tuning phase, we explored two types of kernels (linear and rbf) and tested different values of the regularization parameter C (0.1, 1, and 10).

Once the best parameters were selected for both classifiers, we re-evaluated their performance using the same Stratified 5-Fold strategy, applying the custom *cross_validate_classifier* function, in order to obtain more metrics, for a deep understanding of the results. This part of the project was performed using the dataset without applying any feature selection, dimensionality reduction, or class balancing techniques.

Results & Discussion

Correlation Matrix

The Pearson correlation matrix is presented in Figure 1. The analysis highlights strong correlations between URLLength and NoOFLettersInURL, as well as between DomainTitleMatchScore and URLTitleMatchScore, suggesting possible redundancy among these variables. One variable from each pair was therefore removed to optimise the classification process. On the other hand, several features show low correlation with each other, suggesting that they are independent.

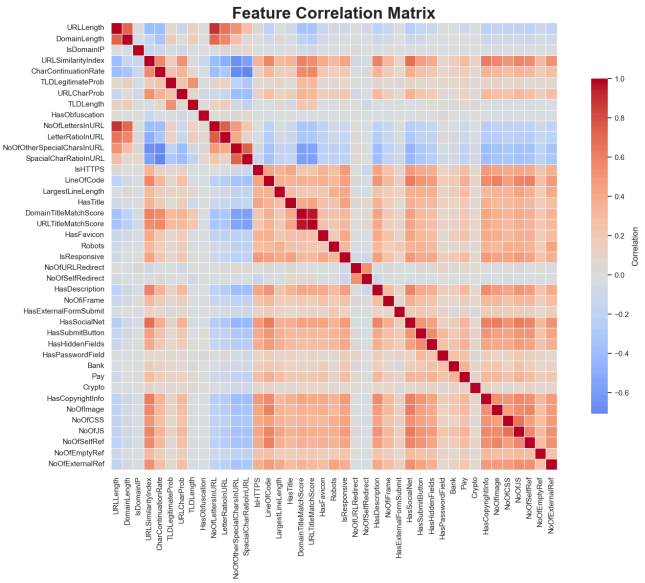

Fig. 1 - Pearson Correlation Matrix between the features of the dataset.

Feature Selection & Dimensionality Reduction

PCA

The first feature reduction technique we applied was PCA, with the aim of reducing the dimensionality of the dataset while retaining most of the variability. The plot representing the variance explained by the components is shown in Figure 2. After applying PCA, 26 components were selected, which explain 95% of the variance, indicating that 26 out of 39 variables are sufficient to capture most of the data's variance.

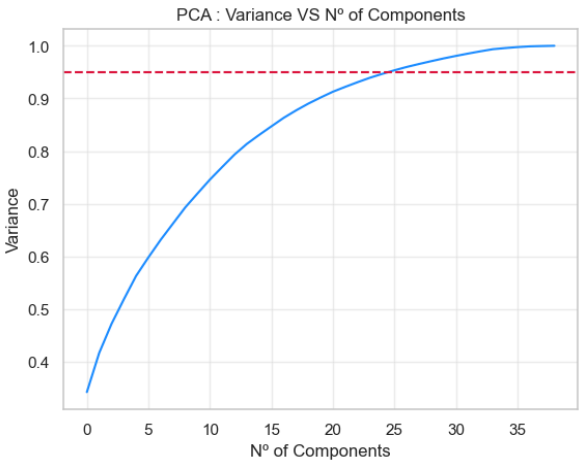

Fig. 2 - PCA plot showing the variance explained by each component of the dataset. The red dashed line limits the 95% of variance.

The plot of the first two principal components (PC1 and PC2) is shown in Figure 3. From this, it was possible to observe that the classes are relatively well separated, suggesting that just the first two components are already effective in capturing a significant portion of the variance in the data. Although the PCA goal is not to separate the classes, it was successful in reducing the dimensionality of the original feature space, while preserving the most important variance-related information, which helps in distinguishing the classes.

We expect that the classifiers will achieve great scores and a good performance in the task of identifying phishing URLs, since the results we obtained showed a solid base for that.

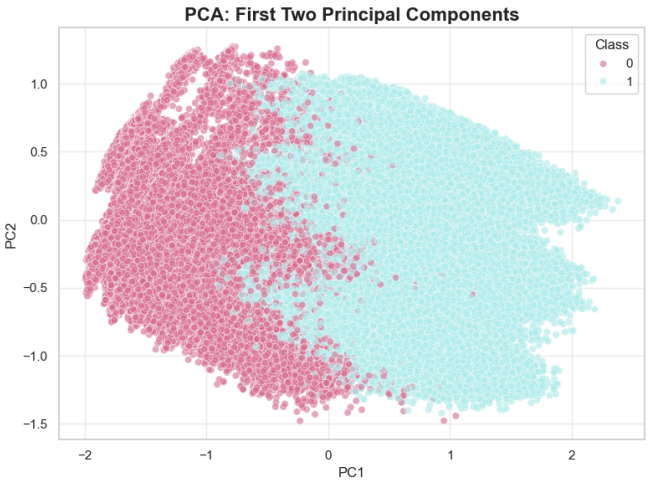

Fig. 3 - First two Principal Components plot, showing the separation between the two classes.

LDA

LDA was the second dimensionality reduction technique we applied. Since we have only two classes, LDA generates only one linear component (LD1), meaning just one axis in the feature space that seeks to maximize the separation between them. The density plot of LD1, shown in Figure 4, illustrates the density curves of both classes. By observing this plot, we can see a clear distinction between the two classes in their distribution along the space. This indicates that LDA was able to find a projection axis in the feature space that maximizes the separation between the classes, concluding that it was effective in reducing the dimensionality of the data while maintaining class discrimination.

Similar to PCA, our results also suggest that the classifiers will achieve good results in the URL classification task, given how distinct the distributions between the classes are.

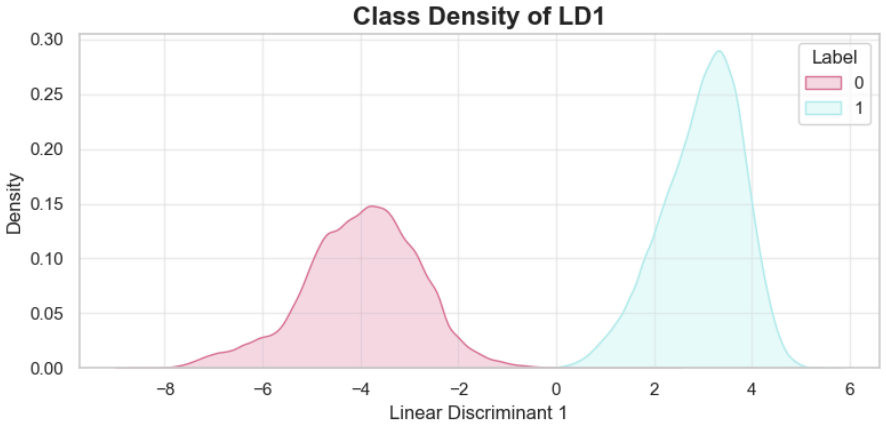

Fig. 4 - Density plot of LD1, showing the density curves of the two classes in the projection space.

Kruskal-Wallis test

The Kruskal-Wallis test was used to assess the statistical significance of each feature in the dataset, with the goal of selecting only those features that showed significant differences between their groups (the classes).

Based on the results obtained, all p-values were extremely low (very close to zero), indicating that all features in the dataset exhibited significant differences between the two classes. The p-values for each feature are presented in Table 1. These results suggest that all variables are important for the classification task in distinguishing between the two classes. Therefore, no feature was considered irrelevant, and the complete dataset was used across the various classifiers. Thus, dimensionality reduction was considered unnecessary in this specific case.

Table 1 P-values for all features used in the classifiers.

Feature	p-value	Feature	p-value
URLLength	0.00000	NoOfCSS	0.00000
DomainLength	0.00000	NoOfJS	0.00000
URLSimilarityIndex	0.00000	NoOfSelfRef	0.00000
CharContinuationRate	0.00000	NoOfEmptyRef	0.00000
TLDLegitimateProb	0.00000	NoOfExternalRef	0.00000
URLCharProb	0.00000	NoOfSelfRedirect	0.00000
LetterRatioInURL	0.00000	IsDomainIP	0.00000
NoOfOtherSpecialCharsInURL	0.00000	HasObfuscation	0.00000
SpacialCharRatioInURL	0.00000	NoOfURLRedirect	0.00000
IsHTTPS	0.00000	TLDLength	0.00000
LineOfCode	0.00000	HasCopyrightInfo	0.00000
LargestLineLength	0.00000	Bank	0.00000
HasTitle	0.00000	Pay	0.00000
DomainTitleMatchScore	0.00000	Crypto	0.00000
HasFavicon	0.00000	HasHiddenFields	0.00000
Robots	0.00000	HasPasswordField	0.00000
IsResponsive	0.00000	HasSubmitButton	0.00000
HasDescription	0.00000	HasSocialNet	0.00000
NoOfiFrame	0.00000	HasExternalFormSubmit	0.00000

Classifiers' results with imbalanced dataset

The first phase of the performance evaluation of the classifiers was done with the preprocessed data, without dimensionality reduction. The results are shown in Table 2. The confusion matrices are presented in the APPENDIX, in Table 13.

Table 2 Performance evaluation of the classifiers with all features.

Classifier	Accuracy	F1-score	Sensitivity	Specificity
MDC (*euclidean dist.*)	97.60 ± 0.07	97.89 ± 0.06	97.09 ± 0.11	98.29 ± 0.14
MDC (*mahalanobis dist.*)	99.93 ± 0.01	99.94 ± 0.01	100 ± 0	99.83 ± 0.03
Fisher-LDA	99.92 ± 0.01	99.93 ± 0.01	100 ± 0	99.82 ± 0.03
KNN	99.75 ± 0.04	99.78 ± 0.03	99.86 ± 0.03	99.60 ± 0.07
GNB	99.97 ± 0.01	99.97 ± 0	99.95 ± 0.01	99.98 ± 0.01
SVM	99.99 ± 0	99.99 ± 0	100 ± 0	99.97 ± 0.01

Even with default parameters in most classifiers (except for KNN, whose *n_neighbors* was set to 3) and without applying feature reduction methods, all models achieved very good results. The SVM, Fisher-LDA, and MDC evaluated with the Mahalanobis distance achieved nearly perfect results, with emphasis on sensitivity of 1.0 and F1 scores above 99.90%. The confusion matrices showed almost no FP for these models. However, due to the higher presence of class 1 compared to class 0, these sensitivity results and FPs may not be reliable. Hence, there is a need to evaluate the model with class balancing.

The MDC with Euclidean distance performed below the other models, with an accuracy of 97.6% and an F1 score below 98%. Although still yielding satisfactory and high results, this classifier showed a higher rate of FP and FN compared to all other models. This suggests that this classifier is not ideal for this type of problem. However, the same model with the Mahalanobis distance showed better results, due to the greater robustness of the distance, which considers the variance and correlation between points, providing a more accurate measure.

The GNB also performed well, considering that this model was only used for comparison, given that the variables do not follow a normal distribution. However, these results suggest that the model may be relatively robust even with violations of its assumptions, possibly due to data characteristics that lead to a high discrimination ability between classes.

To investigate whether dimensionality reduction impacts model performance, we performed the same evaluation with the feature set obtained from PCA and LDA. The results are presented in Table 3 and Table 4, respectively. The confusion matrices are shown in Table 14 and Table 15, respectively, in the APPENDIX.

Table 3 Performance evaluation of the classifiers with dimensionality reduction by PCA.

Classifier	Accuracy	F1-score	Sensitivity	Specificity
MDC (*euclidean dist.*)	97.59 ± 0.07	97.88 ± 0.06	97.08 ± 0.11	98.27 ± 0.14
MDC (*mahalanobis dist.*)	99.89 ± 0.02	99.90 ± 0.02	99.94 ± 0.01	99.81 ± 0.05
Fisher-LDA	99.88 ± 0.02	99.90 ± 0.02	99.95 ± 0.01	99.79 ± 0.04
KNN	99.71 ± 0.04	99.75 ± 0.03	99.81 ± 0.03	99.58 ± 0.07
GNB	95.56 ± 0.07	96.22 ± 0.06	98.51 ± 0.09	91.58 ± 0.21
SVM	99.97 ± 0.01	99.98 ± 0.01	99.99 ± 0	99.94 ± 0.02

Table 4 Performance evaluation of the classifiers with dimensionality reduction by LDA.

Classifier	Accuracy	F1-score	Sensitivity	Specificity
MDC (*euclidean dist.*)	99.93 ± 0.01	99.94 ± 0.01	100 ± 0	99.84 ± 0.03
MDC (*mahalanobis dist.*)	99.93 ± 0.01	99.94 ± 0.01	100 ± 0	99.84 ± 0.03
Fisher-LDA	99.92 ± 0.01	99.93 ± 0.01	100 ± 0	99.82 ± 0.03
KNN	99.97 ± 0.01	99.97 ± 0.01	99.98 ± 0.01	99.94 ± 0.02
GNB	99.97 ± 0.01	99.97 ± 0.01	99.98 ± 0.01	99.95 ± 0.02
SVM	99.97 ± 0.01	99.97 ± 0.01	99.99 ± 0	99.94 ± 0.02

After PCA, there was a slight decrease in the performance of many classifiers. For example, the GNB dropped from a specificity of 99.98% to 91.58%. Although the GNB showed good results even with violations of the normal distribution assumptions in the features, PCA creates new variables that, although uncorrelated, end up violating the assumption that they should be independent. This helps explain the decrease in GNB performance after dimensionality reduction.

The SVM, the MDC with Mahalanobis distance, and Fisher-LDA continued to perform the best overall. Since PCA is an unsupervised technique that only considers the variance explained by the data, it may discard components with low variance but important information for class discrimination. However, as seen in the PCA results, the data structure suggests that the classes can be easily discriminated against, so the good results are expected.

Compared to PCA and even the results with the original features, results improved when LDA was applied as the dimensionality reduction technique. Unlike PCA, LDA is a supervised technique that aims to find a linear combination that best separates the two classes, which explains the almost excellent results in all classifiers, something already anticipated by the LD1 plot, which showed an optimal separation between the classes. After applying this technique, all classifiers maintained or improved their metric results compared to the results when using the original dataset. The F1 score remained above 99.90% for all models, and the number of FP and FN also decreased drastically. The GNB, even though a simpler model, was able to achieve results on the same level as the more complex models.

Overall, the results confirm that the dataset's structure facilitates the classification task for both simple and complex models, especially when supervised dimensionality reduction techniques are applied. The intrinsic structure of the data indicates that the classes are naturally well-separated, which aids discrimination and contributes to the high performances observed.

Classifiers' results with balanced dataset

The next step in the project involved addressing class imbalance using random oversampling and random undersampling, in order to assess the impact of a balanced dataset on model performance. Both class balancing results on the dataset are in the APPENDIX. The results for random oversampling and random undersampling on the feature set without dimensionality reduction are presented in Table 5 and Table 6, respectively. The corresponding confusion matrices can be found in Table 16 and Table 17 in the APPENDIX.

Table 5 Random oversampling results of the performance evaluation of the classifiers with all features.

Classifier	Accuracy	F1-score	Sensitivity	Specificity
MDC (*euclidean dist.*)	97.70 ± 0.04	97.68 ± 0.04	97.10 ± 0.10	98.29 ± 0.06
MDC (*mahalanobis dist.*)	99.93 ± 0.01	99.93 ± 0.01	99.99 ± 0	99.87 ± 0.03
Fisher-LDA	99.93 ± 0.01	99.93 ± 0.01	99.99 ± 0	99.87 ± 0.03
KNN	99.76 ± 0.04	99.76 ± 0.04	99.81 ± 0.02	99.70 ± 0.03
GNB	99.97 ± 0.01	99.97 ± 0.01	99.95 ± 0.01	99.98 ± 0.02
SVM	99.99 ± 0	99.99 ± 0	100 ± 0	99.98 ± 0.01

Table 6 Random undersampling results of the performance evaluation of the classifiers with all features.

Classifier	Accuracy	F1-score	Sensitivity	Specificity
MDC (*euclidean dist.*)	97.68 ± 0.02	97.66 ± 0.02	97.07 ± 0.06	98.29 ± 0.03
MDC (*mahalanobis dist.*)	99.93 ± 0.01	99.93 ± 0.01	99.99 ± 0	99.87 ± 0.02
Fisher-LDA	99.93 ± 0.01	99.93 ± 0.01	99.99 ± 0	99.87 ± 0.02
KNN	99.72 ± 0.02	99.72 ± 0.02	99.81 ± 0.03	99.63 ± 0.02
GNB	99.97 ± 0.01	99.97 ± 0.01	99.95 ± 0.01	99.98 ± 0.01
SVM	99.99 ± 0	99.99 ± 0	100 ± 0	99.97 ± 0.01

The results show that class balancing had some impact on certain classifiers, particularly those that previously exhibited maximum sensitivity due to the higher presence of the positive class (class 1, legitimate URLs) in the original dataset—though sensitivity remained high overall. SVM, Fisher-LDA, and MDC with Mahalanobis distance continued to achieve excellent results, indicating that these models are highly robust for this classification problem, regardless of whether the dataset is balanced or imbalanced.

The MDC with Euclidean distance, which had shown weaker performance compared to the others on the imbalanced dataset, achieved better results after random oversampling, although it still lagged behind the more robust models. Despite these improvements, the number of false positives and false negatives increased. In the case of KNN, random oversampling also led to a slight improvement in the metric values, suggesting that this classifier performs better on balanced datasets. GNB, on the other hand, maintained the same results.

As for random undersampling, it also had some impact on the models. More robust ones, such as SVM and MDC with Mahalanobis distance, remained stable and performed excellently. However, KNN and MDC with Euclidean distance showed decreased performance compared to random oversampling. This may be due to the reduction in sample size, which likely led to a loss of important information for class discrimination, thereby reducing the models' generalization capacity. Still, even with fewer samples, all results remained quite satisfactory, confirming the earlier conclusion that class separability is high due to the dataset's characteristics. Moreover, with random undersampling, the minority class (class 0, phishing URLs) was still well represented, with enough samples for the model to "learn" and classify effectively.

Overall, both random undersampling and random oversampling yielded relatively similar results, suggesting that the models' classification capabilities are robust for this dataset and not highly sensitive to the balancing technique applied.

Dimensionality reduction was also applied to assess its impact on model performance, in order to determine whether reducing the number of features would affect results on a balanced dataset. The outcomes for random oversampling and random undersampling with PCA applied are shown in Table 7 and Table 8, respectively. Results using LDA are presented in Table 9 and Table 10. Confusion matrices are included in Tables 18–21 in the APPENDIX.

Table 7 Random oversampling results of the performance evaluation of the classifiers with dimensionality reduction by PCA.

Classifier	Accuracy	F1-score	Sensitivity	Specificity
MDC (*euclidean dist.*)	97.68 ± 0.05	97.67 ± 0.05	97.08 ± 0.10	98.28 ± 0.06
MDC (*mahalanobis dist.*)	99.85 ± 0.01	99.85 ± 0.01	99.85 ± 0.02	99.85 ± 0.01
Fisher-LDA	99.85 ± 0.01	99.85 ± 0.01	99.85 ± 0.02	99.85 ± 0.01
KNN	99.72 ± 0.02	99.72 ± 0.02	99.76 ± 0.03	99.67 ± 0.02
GNB	95.10 ± 0.12	95.25 ± 0.11	98.39 ± 0.07	91.81 ± 0.24
SVM	99.97 ± 0	99.97 ± 0	99.99 ± 0	99.96 ± 0.01

Table 8 Random undersampling results of the performance evaluation of the classifiers with dimensionality reduction by PCA.

Classifier	Accuracy	F1-score	Sensitivity	Specificity
MDC (*euclidean dist.*)	97.66 ± 0.02	97.65 ± 0.02	97.05 ± 0.06	98.27 ± 0.04
MDC (*mahalanobis dist.*)	99.85 ± 0.01	99.85 ± 0.01	99.85 ± 0.01	99.85 ± 0.02
Fisher-LDA	99.85 ± 0.01	99.85 ± 0.01	99.85 ± 0.01	99.85 ± 0.02
KNN	99.67 ± 0.02	99.67 ± 0.02	99.75 ± 0.03	99.59 ± 0.03
GNB	95.06 ± 0.05	95.22 ± 0.05	98.35 ± 0.06	91.77 ± 0.05
SVM	99.96 ± 0.00	99.96 ± 0.00	99.99 ± 0.00	99.94 ± 0.00

Table 9 Random oversampling results of the performance evaluation of the classifiers with dimensionality reduction by LDA.

Classifier	Accuracy	F1-score	Sensitivity	Specificity
MDC (*euclidean dist.*)	99.91 ± 0.00	99.91 ± 0.00	100 ± 0	99.83 ± 0.02
MDC (*mahalanobis dist.*)	99.91 ± 0.00	99.91 ± 0.00	100 ± 0	99.83 ± 0.02
Fisher-LDA	99.91 ± 0.00	99.91 ± 0.00	100 ± 0	99.83 ± 0.02
KNN	99.95 ± 0.00	99.95 ± 0.00	99.97 ± 0.00	99.94 ± 0.02
GNB	99.96 ± 0.00	99.96 ± 0.00	99.97 ± 0.00	99.95 ± 0.00
SVM	99.96 ± 0.00	99.96 ± 0.00	99.99 ± 0.00	99.94 ± 0.02

Table 10 Random undersampling results of the performance evaluation of the classifiers with dimensionality reduction by LDA.

Classifier	Accuracy	F1-score	Sensitivity	Specificity
MDC (*euclidean dist.*)	99.92 ± 0.00	99.92 ± 0.00	100 ± 0	99.84 ± 0.00
MDC (*mahalanobis dist.*)	99.92 ± 0.00	99.92 ± 0.00	100 ± 0	99.84 ± 0.00
Fisher-LDA	99.92 ± 0.00	99.92 ± 0.00	100 ± 0	99.84 ± 0.00
KNN	99.96 ± 0.00	99.96 ± 0.00	99.98 ± 0.00	99.95 ± 0.00
GNB	99.96 ± 0.00	99.96 ± 0.00	99.98 ± 0.00	99.95 ± 0.00
SVM	99.96 ± 0.00	99.96 ± 0.00	99.99 ± 0.00	99.94 ± 0.00

PCA led to a performance drop in many classifiers, particularly in GNB, which suffered the most significant decrease—especially in specificity, which fell from 99% to 91% across both balancing techniques. In the case of GNB, these results reinforce that PCA violates the assumption of variable independence, resulting in lower scores. The overall performance decline can be attributed to PCA's unsupervised nature. As previously mentioned, PCA may discard components that account for little variance but still carry critical information for class

separability. Nevertheless, more robust models like SVM and KNN maintained performance levels consistent with previous observations.

On the other hand, LDA continued to positively impact the models. As a supervised technique aimed at maximizing class separation, its application to the balanced dataset led to improved metric values across all classifiers, including simpler models such as GNB and MDC with Euclidean distance. All metrics surpassed 99.90%, with sensitivities that were nearly or even completely perfect. There was also a notable decrease in both false positives and false negatives, highlighting LDA as an ideal technique for this dataset. Combined with class balancing, it enabled models to generalize well and achieve excellent classification results.

In summary, class balancing allowed for a more rigorous evaluation of the classifiers. Even though random undersampling reduced the overall number of samples, the remaining data still held enough information to produce excellent results. The high performance observed across the different balancing and dimensionality reduction techniques reinforces that the intrinsic structure of the data and the information provided by the features support clear class separation—resulting in outstanding classification, even with simpler models. Thus, despite PCA having negatively impacted performance, it was still possible to obtain very satisfactory results. LDA, however, proved to be the most effective dimensionality reduction technique, consistently delivering top-tier results.

Hyperparameter Tuning

Even with the excellent results we've achieved so far, we decided to investigate whether the models' performance could be further improved by optimizing certain parameters. This step was carried out solely to explore potential enhancements in classification and to reinforce the robustness of the classifiers. The optimal parameter values obtained are shown in Table 11.

Table 11 Best hyperparameter value achieved in the optimization stage.

Hyperparameter (*classifier*)	Best Value
n_neighbors (KNN)	3
kernel (SVM)	linear
c (SVM)	10

For KNN, the grid search results across several odd values of *n_neighbors* were particularly interesting: the best-performing value was 3 — the same we have been using up until now. This outcome reinforces our ongoing observation that the classes in the original dataset are already well separated in feature space, which allows even a small number of "nearest neighbors" to achieve accurate classifications. Higher values of the parameter would lead to a greater number of neighbors being considered when classifying a point, increasing the likelihood of including neighbors from the opposite class near class boundaries, and thereby slightly reducing performance.

The metric results are consistent with what we had previously observed and match those in Table 2, since the model itself hasn't changed. Additionally, the ROC curve showed a maximum AUC. Altogether, these results confirm both the robustness of the KNN model and the high degree of class separability in the dataset.

For SVM, the best performance was achieved using a linear kernel and a regularization parameter of C = 10. These results suggest that the classes are linearly separable in feature space, a conclusion supported by our LDA findings — which demonstrate clear class separation — and by the metrics from the various classifiers. The selection of a relatively high regularization parameter indicates that the model prioritized minimizing classification errors during training, even at the cost of a smaller margin. Still, due to the distribution of the classes, this value of C allows the model to perform at a very high level.

As shown in Table 12, the metric results are consistent with previous findings, with the model reaching nearly perfect — or indeed perfect — values, all at or above 99.99%, particularly in sensitivity and the maximum ROC AUC. While the SVM had already shown excellent performance, this parameter optimization revealed which values could lead to even more accurate classification.

Table 12 Results of the performance evaluation of the SVM with kernel = linear and c = 10.

Classifier	Accuracy	F1-score	Sensitivity	Specificity
SVM	99.99 ± 0.00	99.99 ± 0.00	100 ± 0	99.99 ± 0.00

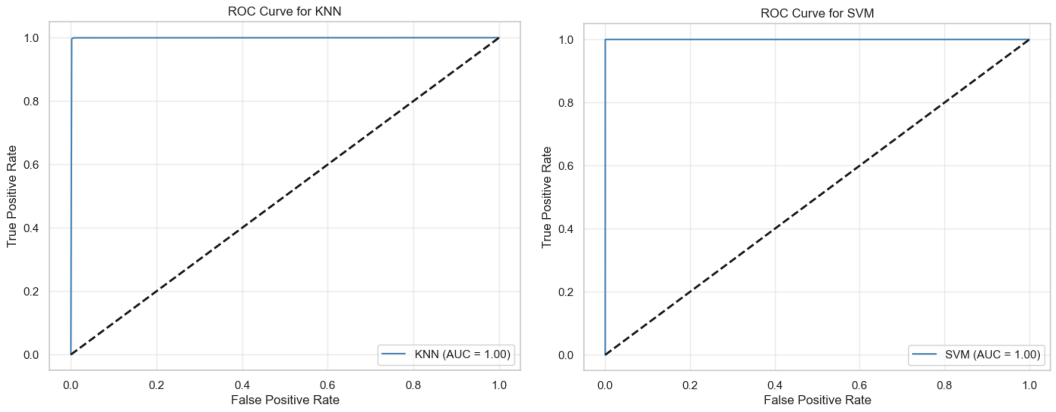

Fig. 5 - ROC curves of the best-performing models. KNN curve (left) and SVM curve (right).

Conclusion

In summary, the results obtained in all phases of this project were excellent, demonstrating the robustness of the classifiers in identifying and distinguishing legitimate URLs from phishing URLs. The outcomes of the dimensionality reduction techniques, the metric values, and the optimized parameters confirm that the dataset contains features with information that allows for easy discrimination between classes, which are linearly separable in the feature space. As such, these conclusions support the fact that both simple and more complex models perform very well.

Balancing techniques had some impact on model performance; however, the models remained consistent, with balancing mainly providing a more rigorous evaluation. Regarding dimensionality reduction techniques, PCA slightly reduced the performance of the classifiers, as it only considers the variance explained by the features—potentially discarding important information for class discrimination. On the other hand, LDA, which aims to maximize class separation, yielded the best results in this project and is therefore an ideal technique to apply to this dataset.

References

[1] *A Survey of Preprocessing Methods Used for Analysis of Big Data Originated From Smart Grids*. (n.d.). Ieeexplore.ieee.org. https://ieeexplore.ieee.org/document/9730885

[2] Bhavsar, V., Kadlak, A., & Sharma, S. (2018). Study on Phishing Attacks. *International Journal of Computer Applications*, *182*(33), 27–29. http://dx.doi.org/10.5120/ijca2018918286

[3] BÜYÜKKEÇECİ, M., & OKUR, M. C. (2022). A Comprehensive Review of Feature Selection and Feature Selection Stability in Machine Learning. *GAZI UNIVERSITY JOURNAL of SCIENCE*. https://doi.org/10.35378/gujs.993763

[4] Duaa Alrufaihi, Omogbai Oleghe, Almanei, M., Sandeep Jagtap, & Konstantinos Salonitis. (2022). Feature Reduction and Selection for Use in Machine Learning for Manufacturing. *19th International Conference in Manufacturing Research ICMR 2022*. https://doi.org/10.3233/ATDE220605

[5] García, S., Ramírez-Gallego, S., Luengo, J., Benítez, J. M., & Herrera, F. (2016). Big data preprocessing: methods and prospects. *Big Data Analytics*, *1*(1). https://doi.org/10.1186/s41044-016-0014-0

[6] Ma, Y., Jiao, L., Liu, F., Yang, S., & Chen, P. (2023). Feature Distribution Representation Learning Based on Knowledge Transfer for Long-Tailed Classification. *IEEE Transactions on Multimedia*, *PP*(99), 1–14. https://doi.org/10.1109/TMM.2023.3303697

[7] Maharana, K., Mondal, S., & Nemade, B. (2022). A Review: Data Pre-Processing and Data Augmentation Techniques. *Global Transitions Proceedings*, *3*(1), 91–99. Sciencedirect. https://www.sciencedirect.com/science/article/pii/S2666285X22000565

[8] Prasad, A., & Chandra, S. (2023). PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning. *Computers & Security*, *136*, 103545. https://doi.org/10.1016/j.cose.2023.103545

[9] Singh, A., & Tiwari, A. (2021). A Study of Feature Selection and Dimensionality Reduction Methods for Classification-Based Phishing Detection System. *International Journal of Information Retrieval Research*, *11*(1), 1–35. https://doi.org/10.4018/ijirr.2021010101

APPENDIX

Feature Distributions

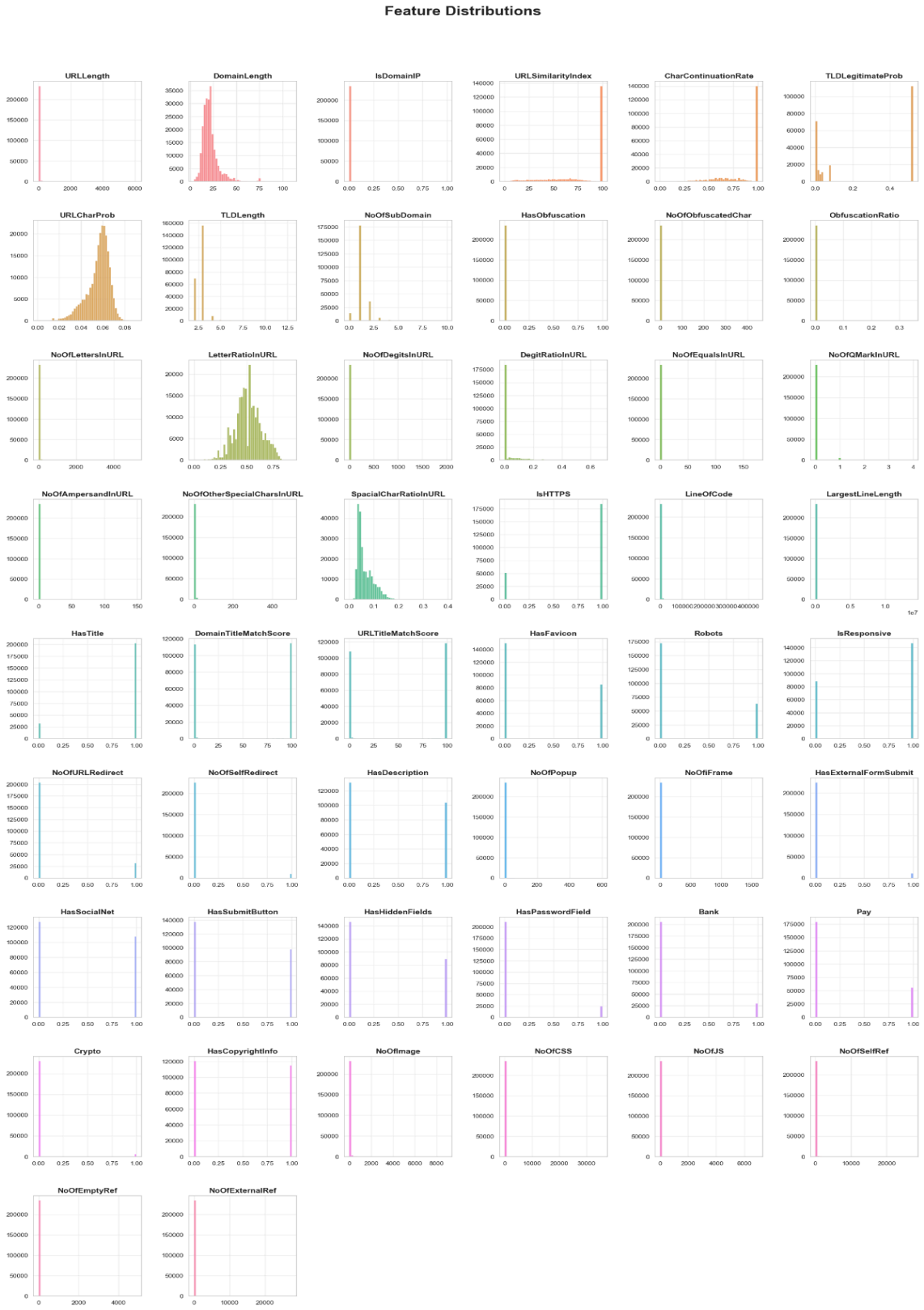

Fig. 6 - Histograms showing the distribution of each feature of the dataset.

Fig. 7 - Box plots showing the distribution for each class of each non binary feature of the dataset.

Class Balancing

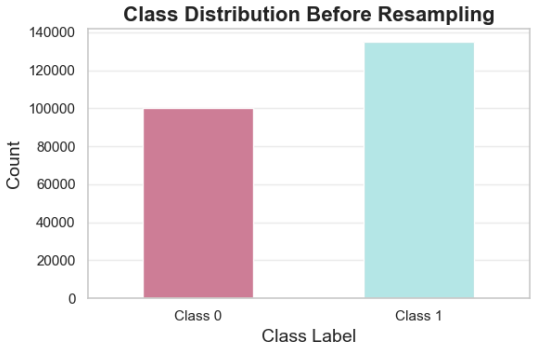

Fig. 8 - Class distribution of the dataset before applying any resample methods.

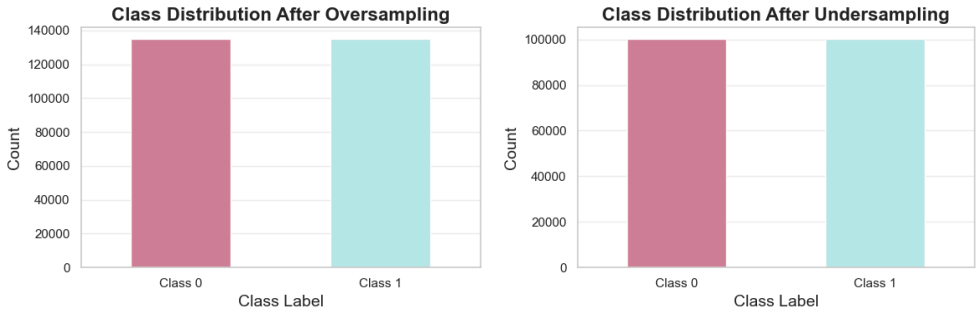

Fig. 9 - Class distribution of the dataset after applying random oversampling (left) and random undersampling (right).

Confusion Matrices

Table 13 Confusion matrix of the classifiers with all features.

Classifier	TN	FN	FP	TP
MDC (*euclidean dist.*)	98421	1716	3922	130928
MDC (*mahalanobis dist.*)	99971	166	0	134850
Fisher-LDA	99957	180	0	134850
KNN	99740	397	193	134657
GNB	100119	18	64	134786
SVM	100109	28	0	134850

Table 14 Confusion matrix of the classifiers with dimensionality reduction by PCA.

Classifier	TN	FN	FP	TP
MDC (*euclidean dist.*)	98406	1731	3941	130909
MDC (*mahalanobis dist.*)	99942	195	75	134775
Fisher-LDA	99923	214	61	134789
KNN	99715	422	256	134594
GNB	91707	8430	2009	132841
SVM	100079	58	9	134841

Table 15 Confusion matrix of the classifiers with dimensionality reduction by LDA.

Classifier	TN	FN	FP	TP
MDC (*euclidean dist.*)	99973	164	0	134850
MDC (*mahalanobis dist.*)	99973	164	0	134850
Fisher-LDA	99958	179	0	134850
KNN	100078	59	23	134827
GNB	100085	52	24	134826
SVM	100072	65	9	134841

Table 16 Confusion matrix of the classifiers with all features after random oversampling.

Classifier	TN	FN	FP	TP
MDC (*euclidean dist.*)	132550	2300	3914	130936
MDC (*mahalanobis dist.*)	134669	181	10	134840
Fisher-LDA	134669	181	10	134840
KNN	134449	401	259	134591
GNB	134824	26	65	134785
SVM	134821	29	0	134850

Table 17 Confusion matrix of the classifiers with all features after random undersampling.

Classifier	TN	FN	FP	TP
MDC (*euclidean dist.*)	98421	1716	2939	97198
MDC (*mahalanobis dist.*)	100006	131	6	100131
Fisher-LDA	100006	131	6	100131
KNN	99765	372	188	99949
GNB	100119	18	52	100085
SVM	100109	28	0	100137

Table 18 Confusion matrix of the classifiers with dimensionality reduction by PCA after random oversampling.

Classifier	TN	FN	FP	TP
MDC (*euclidean dist.*)	132532	2318	3937	130913
MDC (*mahalanobis dist.*)	134646	204	207	134643
Fisher-LDA	134646	204	207	134643
KNN	134411	439	320	134530
GNB	123803	11047	2176	132674
SVM	134791	59	13	134837

Table 19 Confusion matrix of the classifiers with dimensionality reduction by PCA after random undersampling.

Classifier	TN	FN	FP	TP
MDC (*euclidean dist.*)	98404	1733	2954	97183
MDC (*mahalanobis dist.*)	99990	147	151	99986
Fisher-LDA	99990	147	151	99986
KNN	99724	413	248	99889
GNB	91900	8237	1655	98482
SVM	100079	58	13	100124

Table 20 Confusion matrix of the classifiers with dimensionality reduction by LDA after random oversampling.

Classifier	TN	FN	FP	TP
MDC (*euclidean dist.*)	134620	230	0	134850
MDC (*mahalanobis dist.*)	134620	230	0	134850
Fisher-LDA	134620	230	0	134850
KNN	134774	76	46	134804
GNB	134779	71	34	134816
SVM	134765	85	14	134836

Table 21 Confusion matrix of the classifiers with dimensionality reduction by LDA after random undersampling.

Classifier	TN	FN	FP	TP
MDC (*euclidean dist.*)	99972	165	0	100137
MDC (*mahalanobis dist.*)	99972	165	0	100137
Fisher-LDA	99972	165	0	100137
KNN	100082	55	25	100112
GNB	100087	50	24	100113
SVM	100076	61	10	100127